EVOLUTIONARY COMPUTATION

LABORATORY 01

Greedy heuristics

Selecting and arranging a fixed-size subset of nodes

ASSIGNMENT 1 · REPORT

Authors

Paweł Świątek
160301

Jędrzej Szmidt
160292

CORE OBJECTIVE

Minimize cycle length + selected node costs

Every solution selects exactly 50% of the instance nodes, rounded up,
and forms a Hamiltonian cycle through the selected subset.

INSTANCES

TSPA · TSPB

200 nodes each · 100 selected

METHODS

4 constructive strategies

200 evaluated solutions per method

Poznań University of Technology · Artificial Intelligence

October 2026

Problem and experimental design

What is being optimized?

Each instance contains nodes with integer coordinates and a selection cost. The solver must choose a fixed-size subset and order it into a Hamiltonian cycle. The optimization methods use the precomputed distance matrix and node costs.

OBJECTIVE FUNCTION

F(C) = total cycle length + total cost of selected nodes

CONSTRAINTS

- select ceil(n / 2) nodes
- visit every selected node once
- return to the first node
- node indices start at 0

DISTANCES

Euclidean distance rounded mathematically to an integer before optimization begins.

Experimental setup

Instance	Nodes	Selected	Runs / method
TSPA	200	100	200 (one per start node)
TSPB	200	100	200 (one per start node)

Verification and implementation

Implementation

C++17 implementation using standard library containers and a precomputed symmetric distance matrix. The four methods are evaluated on both supplied instances.

Solution checker

All generated solutions were checked against the provided solution checker, including the saved best and worst solutions reported in this document.

Algorithms I — baseline constructions

The move is selected by objective impact, not distance alone.

Random solution

A baseline that samples a valid subset and randomizes its cyclic order.

```
all_nodes <- [0, 1, ..., n - 1]
shuffle(all_nodes)
route <- first ceil(n / 2) nodes
append route[0] to close the cycle
return route and F(route)
```

Nearest neighbor — end

Builds a path by appending the best unused node to the current end.

```
path <- [start]
while |path| < k:
 for each unused node v:
 delta <- d(last, v) + cost(v)
 v* <- argmin(delta)
 append v* to path
close path and evaluate F(path)
```

Shared decision rule

At every step

candidate changes are compared using the change in the complete objective: distance contribution plus the cost of the newly selected node. This makes the methods adapted to the assignment rather than literal geometric nearest-neighbor rules.

Complexity intuition

The append-only method evaluates unused nodes at one position. The insertion-based methods evaluate multiple positions and therefore require more computation, but can produce better cycles.

Algorithms II — insertion constructions

Both methods evaluate the complete candidate insertion delta.

Nearest neighbor — flexible

Builds an open path and allows insertion at the beginning, end, or any internal gap.

```
path <- [start]
while |path| < k:
 for each unused v:
 evaluate beginning and end
 evaluate every internal gap
 delta includes cost(v)
 apply the best global insertion
close with path[0]
```

Greedy cycle

Maintains a closed cycle and inserts the best unused node into the best cycle edge.

```
cycle <- [start, best second node]
while |cycle| < k:
 for each unused v and edge (a, b):
 delta <- d(a,v) + d(v,b)
 - d(a,b) + cost(v)
 insert the best (v, edge)
return closed cycle and F(cycle)
```

Why the two insertion methods differ

Flexible path

The two ends are open while the path is being built. A beginning or end insertion can change the eventual closing edge.

Greedy cycle

The closing edge is always part of the current cycle, so every candidate is evaluated against every cycle edge.

The distinction is visible in the results: the flexible and greedy-cycle solution spaces are no longer identical.

Results at a glance

Each value is the objective F(C); lower is better.

TSPA

200 evaluated solutions per method

Method	Min	Max	Average	SD
Random	233130.00	289417.00	260294.40	9730.46
NN end	84362.00	90075.00	87476.63	1328.22
NN flexible	72328.00	77022.00	74346.97	981.62
Greedy cycle	72590.00	75281.00	73793.41	527.45

Runtime (ms)

Random solution: 3 ms
Nearest neighbor — end: 84 ms
Nearest neighbor — flexible: 7174 ms
Greedy cycle: 7596 ms

TSPB

200 evaluated solutions per method

Method	Min	Max	Average	SD
Random	184618.00	235381.00	206384.53	8859.90
NN end	56007.00	63166.00	59397.34	2093.12
NN flexible	52915.00	66164.00	59480.28	2171.96
Greedy cycle	55655.00	63734.00	59649.91	2804.92

Runtime (ms)

Random solution: 2 ms
Nearest neighbor — end: 82 ms
Nearest neighbor — flexible: 7306 ms
Greedy cycle: 7633 ms

Main observations

1. Greedy constructions are dramatically better than random selection on both instances.
2. The flexible insertion heuristic and greedy cycle now produce different statistics, as required by their different partial-solution models.
3. Flexible insertion is more computationally expensive than append-only nearest neighbor because it evaluates many more candidate positions.

Best average by instance

Instance	Method	Average	Minimum
TSPA	Greedy cycle	73793.41	72590.00
TSPB	Nearest neighbor — end	59397.34	56007.00

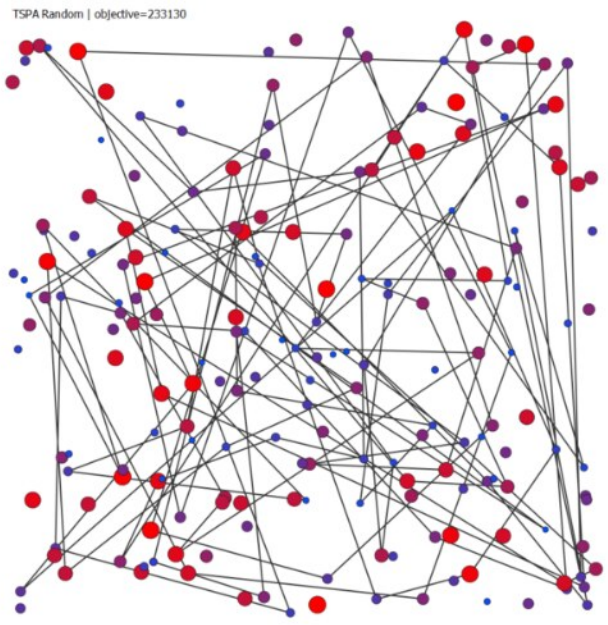
TSPA — best solutions

Node colour and size indicate node cost; the dark line is the selected cycle.

Random solution

best route

objective 233130.00



Stored node-index route

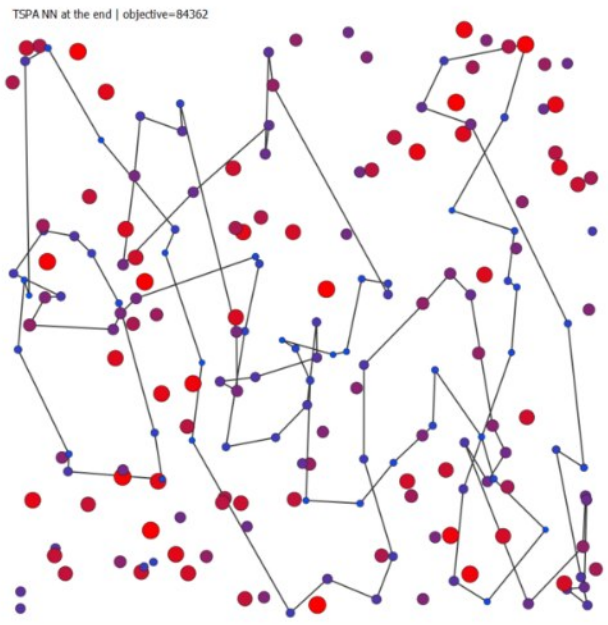
153 24 79 88 138 35 148 186 184 144 81 166 27 83
49 69 114 126 17 131 20 134 91 169 0 61 150 76
143 52 5 2 25 156 107 66 16 18 145 127 135 82 125
63 139 198 181 43 154 11 183 111 93 39 189 100
101 68 191 129 164 130 105 192 51 26 122 167 110
87 99 14 23 193 155 136 92 40 45 84 62 4 7 13 103
8 33 3 197 15 159 1 78 42 196 102 176 58 54 55
153

100 unique selected nodes · closed route

Nearest neighbor — end

best route

objective 84362.00



Stored node-index route

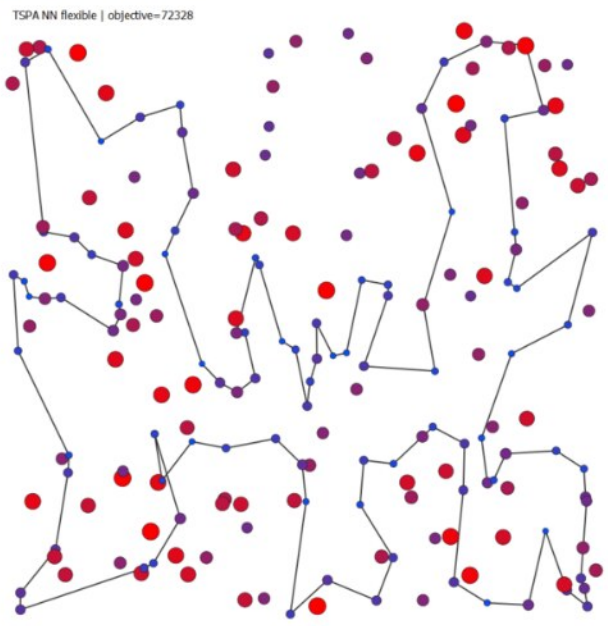
54 9 33 36 177 67 144 53 135 58 134 176 174 106
89 107 57 182 151 93 50 3 1 164 178 29 136 186
184 47 159 130 153 78 88 193 170 146 75 64 150 42
113 43 85 142 8 131 84 158 192 181 114 141 180
132 76 80 48 31 63 149 168 97 127 187 30 123 152
108 60 116 49 11 119 160 83 90 51 16 35 87 45 140
52 179 185 28 102 156 157 71 91 38 100 4 112 139
115 5 54

100 unique selected nodes · closed route

Nearest neighbor — flexibl

best route

objective 72328.00



Stored node-index route

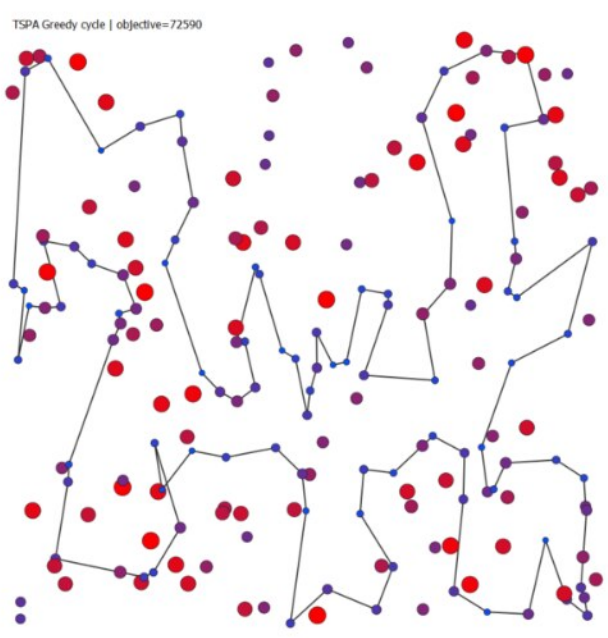
104 129 69 127 97 168 116 149 63 11 49 160 83 123
100 152 108 60 31 48 80 91 157 71 4 76 132 180 28
102 185 35 156 51 16 67 144 140 53 179 52 177 36
33 9 54 131 174 8 164 186 136 163 29 101 178 1
169 3 50 147 47 93 106 42 89 113 130 159 146 170
153 193 78 88 107 64 57 182 151 150 176 72 134 84
58 158 192 181 114 135 73 45 87 141 187 30 189 2
171 104

100 unique selected nodes · closed route

Greedy cycle

best route

objective 72590.00



Stored node-index route

48 80 91 157 71 4 76 132 180 28 102 185 35 156 51
16 67 144 140 53 179 52 177 36 33 9 54 131 174 8
142 164 186 136 163 29 101 178 1 169 3 50 147 47
93 106 42 89 113 130 159 146 170 153 193 78 88
107 64 57 182 151 150 176 72 134 84 58 158 192
181 114 135 73 45 87 141 187 30 189 2 171 69 127
97 160 83 123 90 100 152 108 60 49 11 63 168 149
116 31 48

100 unique selected nodes · closed route

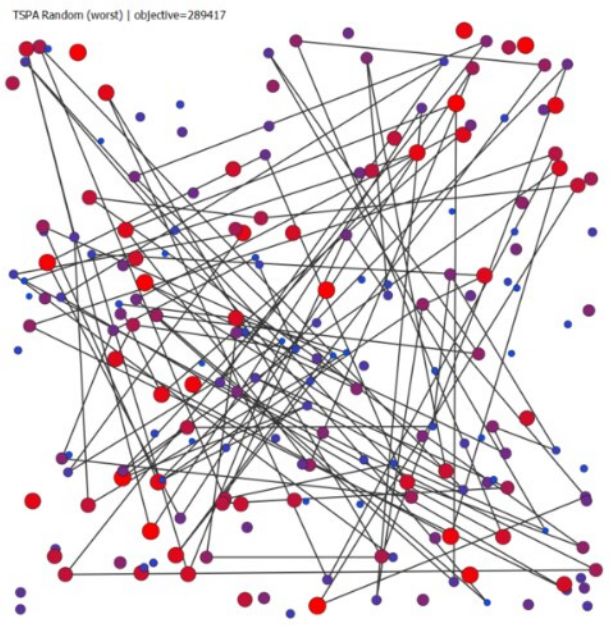
TSPA — worst solutions

Node colour and size indicate node cost; the dark line is the selected cycle.

Random solution

worst route

objective 289417.00



Stored node-index route

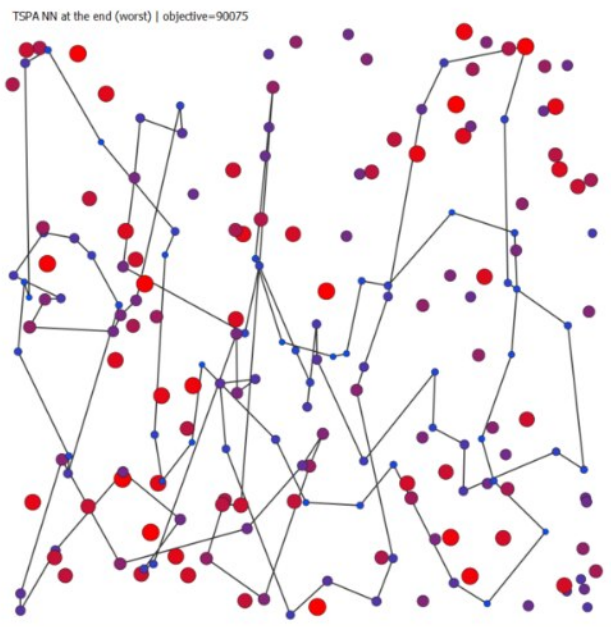
```
109 161 155 165 12 148 144 51 160 95 65 139 167
189 24 61 14 195 182 90 184 23 123 172 36 120 108
199 198 186 192 194 187 140 125 145 176 142 130
43 113 68 0 154 54 143 96 103 185 151 7 27 134
107 115 81 162 57 31 13 75 66 133 6 118 146 8 128
152 183 67 127 173 116 158 10 156 99 38 163 110
26 28 136 49 88 119 46 181 137 20 55 40 82 177
166 17 175 124 149 109
```

100 unique selected nodes · closed route

Nearest neighbor — end

worst route

objective 90075.00



Stored node-index route

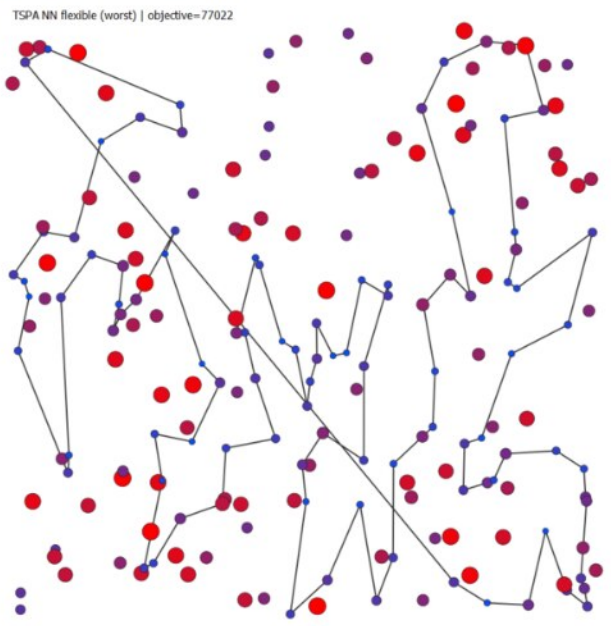
```
5 51 67 177 36 33 9 164 1 50 93 106 89 107 57 182
134 58 135 45 180 141 187 30 132 76 80 48 31 63
149 168 97 127 123 152 108 60 116 49 11 119 160
83 90 157 71 91 38 100 35 16 144 53 140 52 179 84
174 176 150 151 130 159 47 3 178 29 136 186 54
131 99 158 192 181 114 87 28 185 102 156 2 171
189 161 69 104 129 95 92 98 73 41 61 25 10 27 96
70 5
```

100 unique selected nodes · closed route

Nearest neighbor — flexibl

worst route

objective 77022.00



Stored node-index route

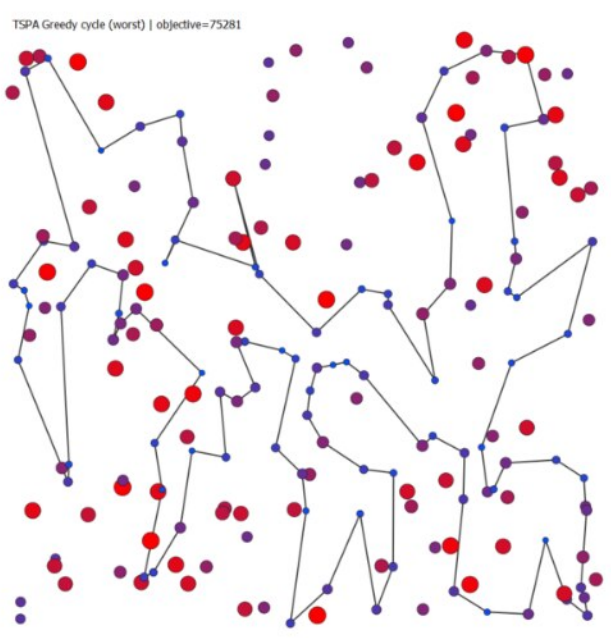
```
31 48 157 71 91 80 108 60 116 149 63 168 95 127
97 49 152 100 123 160 83 90 76 132 180 28 141 30
187 171 2 189 96 87 45 185 35 156 51 16 67 144
140 53 179 52 177 36 33 54 9 131 84 41 73 135 114
181 58 192 158 134 72 176 174 8 142 85 164 186
136 163 29 101 178 1 169 3 50 147 47 93 106 150
151 42 89 113 130 159 146 170 153 193 78 88 107
64 57 182 31
```

100 unique selected nodes · closed route

Greedy cycle

worst route

objective 75281.00



Stored node-index route

```
18 16 52 33 9 54 174 8 142 164 186 136 163 29 101
178 1 169 3 50 147 47 93 106 42 89 113 130 159
146 170 153 193 78 88 107 64 57 182 151 150 176
72 131 36 177 179 53 140 41 84 134 158 192 58 181
114 135 73 45 144 67 35 156 185 102 28 87 141 189
2 171 187 30 180 90 83 160 123 100 152 49 97 127
168 63 149 116 60 108 31 48 80 91 157 71 4 132 76
51 18
```

100 unique selected nodes · closed route

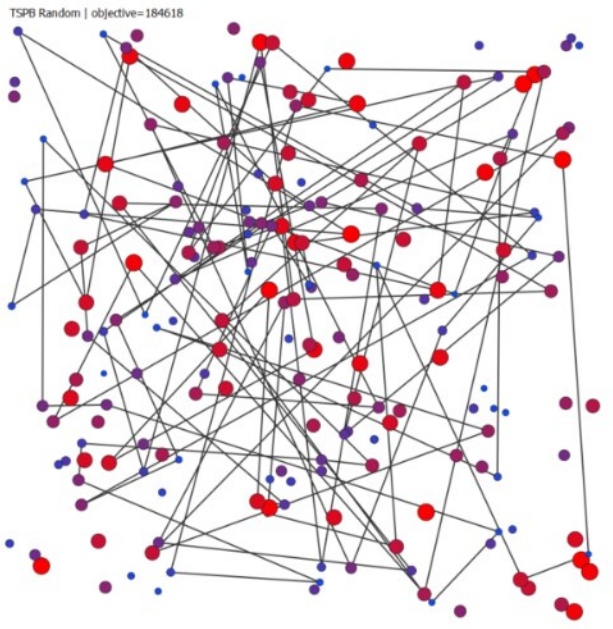
TSPB — best solutions

Node colour and size indicate node cost; the dark line is the selected cycle.

Random solution

best route

objective 184618.00



Stored node-index route

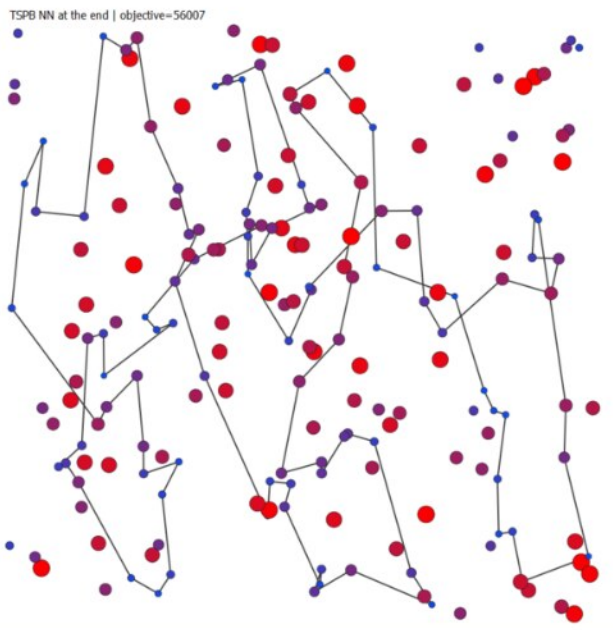
92 46 8 125 71 16 173 143 7 55 28 70 26 40 36 10
164 154 37 0 145 65 191 138 177 51 107 128 113 22
48 176 89 169 39 124 77 34 148 86 165 18 153 4
140 52 142 61 103 161 171 20 115 178 149 31 152 6
197 87 45 156 193 110 2 76 163 85 122 157 99 15
185 179 95 189 116 104 114 192 88 50 196 137 3
117 33 66 121 69 198 83 123 54 187 38 194 159 158
147 92

100 unique selected nodes · closed route

Nearest neighbor — end

best route

objective 56007.00



Stored node-index route

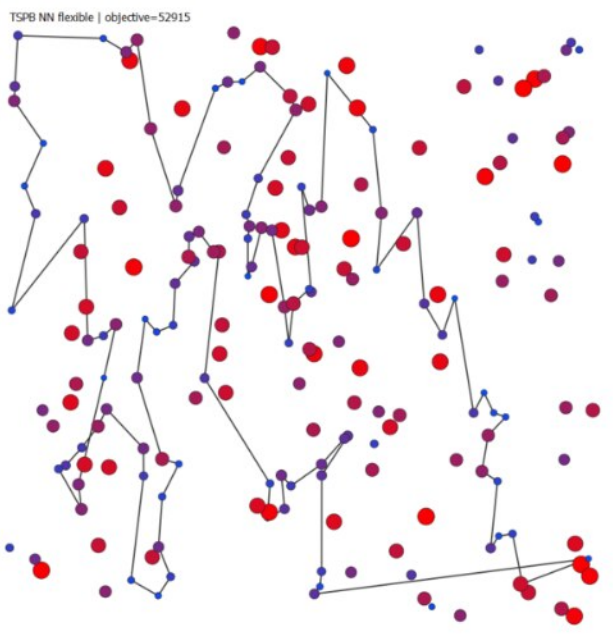
89 57 76 54 127 129 90 153 20 14 2 110 118 56 114
196 10 148 143 78 192 162 104 185 184 3 151 69
172 133 120 96 121 198 48 72 46 86 117 160 136 8
21 154 140 47 170 125 177 6 132 29 197 175 134 45
60 40 71 155 97 115 180 65 28 149 145 158 116 147
42 50 130 30 183 38 64 85 59 167 51 87 179 82 88
171 99 67 24 34 181 152 73 98 27 135 75 79 156 43
89

100 unique selected nodes · closed route

Nearest neighbor — flexibl

best route

objective 52915.00



Stored node-index route

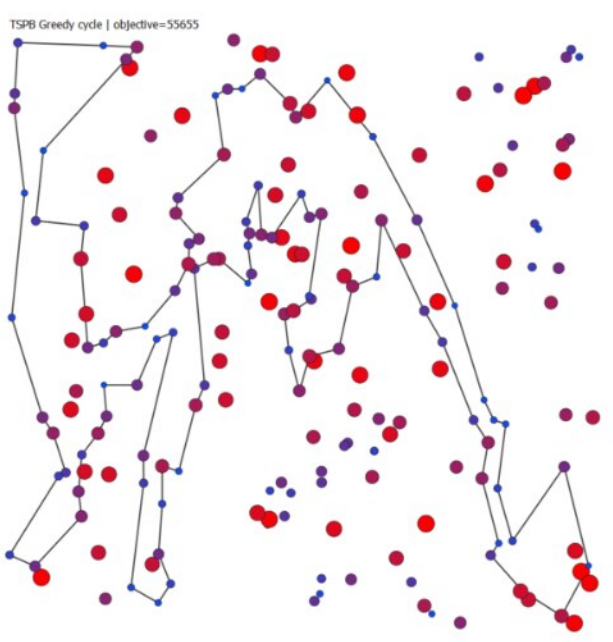
110 2 14 20 74 153 182 52 90 129 127 80 54 192
162 104 76 185 57 89 86 46 72 3 184 173 151 122
117 8 160 69 172 136 133 120 156 43 48 96 198 121
130 191 50 42 147 116 37 146 13 149 28 145 65 158
177 125 26 170 197 36 132 29 6 180 115 155 71 175
134 45 157 60 40 141 97 154 140 47 38 21 176 30
183 189 168 64 59 85 51 73 167 152 34 24 181 179
87 82 110

100 unique selected nodes · closed route

Greedy cycle

best route

objective 55655.00



Stored node-index route

174 36 197 6 180 115 170 97 140 47 155 71 175 134
45 157 60 141 40 33 64 21 189 168 69 160 172 136
133 120 8 117 72 46 86 184 3 173 122 151 98 111
27 135 76 185 162 192 80 52 182 20 74 2 93 110
118 14 153 90 129 127 54 104 57 89 156 43 48 96
198 121 63 130 191 183 30 176 38 154 26 125 177
138 158 145 149 147 42 116 37 146 13 28 65 178
103 29 132 1 174

100 unique selected nodes · closed route

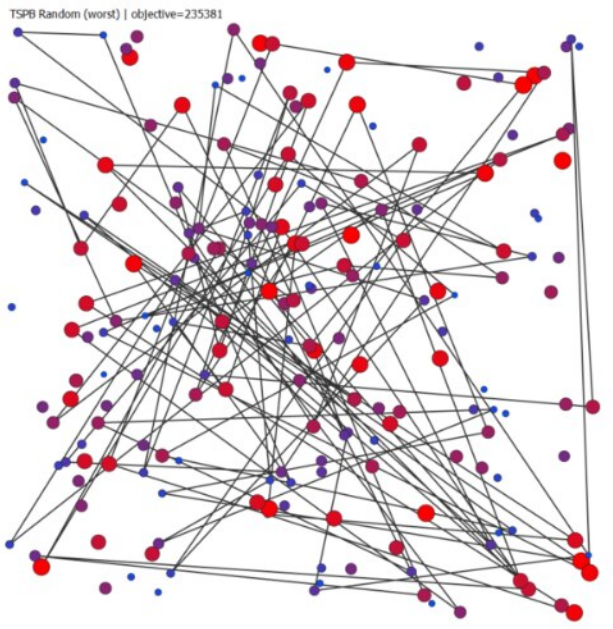
TSPB — worst solutions

Node colour and size indicate node cost; the dark line is the selected cycle.

Random solution

worst route

objective 235381.00



Stored node-index route

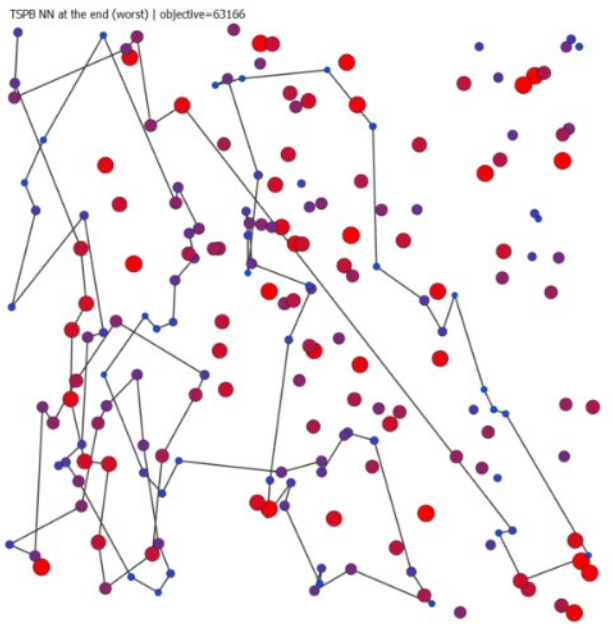
```
26 13 173 63 41 121 183 162 45 144 72 29 73 169
44 64 131 94 112 125 12 123 9 174 39 117 37 116
167 161 11 158 34 22 130 4 32 103 77 25 30 133 59
157 129 180 99 195 138 146 79 78 75 154 18 1 38 2
128 83 52 60 100 28 68 85 109 33 43 71 58 110 82
24 168 108 166 54 141 14 124 185 188 95 49 47 111
199 164 113 23 55 139 53 17 176 67 74 86 10 26
```

100 unique selected nodes · closed route

Nearest neighbor — end

worst route

objective 63166.00



Stored node-index route

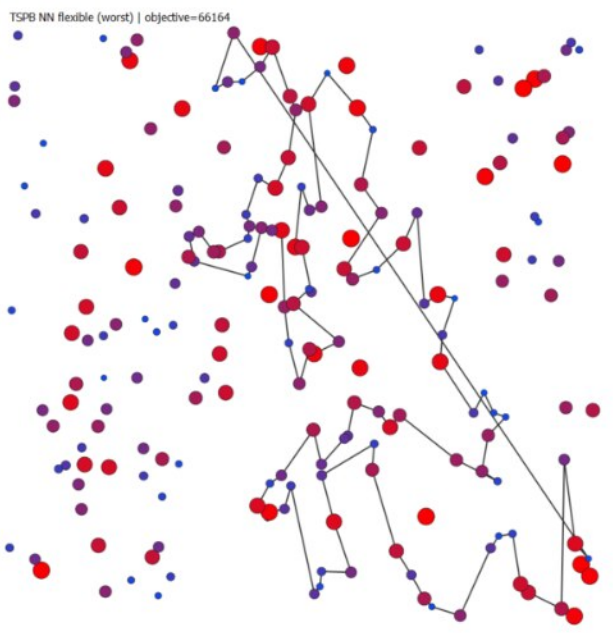
```
14 20 2 110 90 129 127 54 192 162 76 57 89 96 121
198 120 69 172 133 136 160 184 3 151 59 85 167 51
87 179 82 88 171 99 67 24 34 181 152 73 40 60 71
170 154 140 47 38 21 30 183 130 191 116 149 28
145 65 158 125 177 6 132 29 197 175 134 45 157
155 97 115 180 36 1 174 178 103 19 26 64 33 141
142 62 91 113 5 102 53 55 138 146 37 13 147 42 50
195 14
```

100 unique selected nodes · closed route

Nearest neighbor — flexibl

worst route

objective 66164.00



Stored node-index route

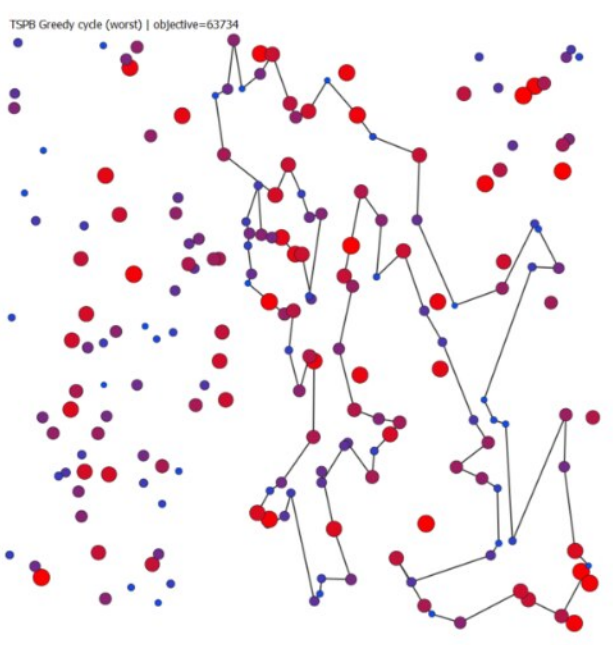
```
110 68 118 93 9 2 188 14 20 74 17 171 39 99 163
31 67 181 126 88 179 87 82 167 51 101 85 107 59
73 25 152 34 24 11 66 119 84 150 153 182 52 90
129 127 80 106 192 54 159 162 104 144 76 135 75
185 79 57 83 89 32 86 46 72 109 137 3 184 173 27
22 111 98 151 122 95 117 8 160 69 21 30 183 189
168 172 136 133 120 92 166 156 43 164 48 96 198
121 58 110
```

100 unique selected nodes · closed route

Greedy cycle

worst route

objective 63734.00



Stored node-index route

```
163 39 171 17 188 2 9 93 110 68 118 56 14 90 129
127 10 148 114 196 78 54 104 77 57 83 89 32 156
43 164 48 96 58 198 121 63 92 166 72 46 86 184 3
137 109 95 117 8 120 133 136 172 160 69 122 173
151 98 111 22 25 73 59 107 85 51 167 82 87 179 88
126 152 181 34 24 31 67 119 84 66 11 27 135 75 79
185 76 144 162 192 80 52 150 182 153 20 74 99 163
```

100 unique selected nodes · closed route

TSPA — best route lists

Routes are listed using zero-based node indices; the first node is repeated at the end to show closure.

Random solution

100 selected nodes · 101 entries including closure

```
153 24 79 88 138 35 148 186 184 144 81 166 27 83 49 69 114
126 17 131 20 134 91 169 0 61 150 76 143 52 5 2 25 156 107
66 16 18 145 127 135 82 125 63 139 198 181 43 154 11 183
111 93 39 189 100 101 68 191 129 164 130 105 192 51 26 122
167 110 87 99 14 23 193 155 136 92 40 45 84 62 4 7 13 103
8 33 3 197 15 159 1 78 42 196 102 176 58 54 55 153
```

Nearest neighbor — end

100 selected nodes · 101 entries including closure

```
54 9 33 36 177 67 144 53 135 58 134 176 174 106 89 107 57
182 151 93 50 3 1 164 178 29 136 186 184 47 159 130 153 78
88 193 170 146 75 64 150 42 113 43 85 142 8 131 84 158 192
181 114 141 180 132 76 80 48 31 63 149 168 97 127 187 30
123 152 108 60 116 49 11 119 160 83 90 51 16 35 87 45 140
52 179 185 28 102 156 157 71 91 38 100 4 112 139 115 5 54
```

Nearest neighbor — flexible

100 selected nodes · 101 entries including closure

```
104 129 69 127 97 168 116 149 63 11 49 160 83 123 100 152
108 60 31 48 80 91 157 71 4 76 132 180 28 102 185 35 156
51 16 67 144 140 53 179 52 177 36 33 9 54 131 174 8 164
186 136 163 29 101 178 1 169 3 50 147 47 93 106 42 89 113
130 159 146 170 153 193 78 88 107 64 57 182 151 150 176 72
134 84 58 158 192 181 114 135 73 45 87 141 187 30 189 2
171 104
```

Greedy cycle

100 selected nodes · 101 entries including closure

```
48 80 91 157 71 4 76 132 180 28 102 185 35 156 51 16 67
144 140 53 179 52 177 36 33 9 54 131 174 8 142 164 186 136
163 29 101 178 1 169 3 50 147 47 93 106 42 89 113 130 159
146 170 153 193 78 88 107 64 57 182 151 150 176 72 134 84
58 158 192 181 114 135 73 45 87 141 187 30 189 2 171 69
127 97 160 83 123 90 100 152 108 60 49 11 63 168 149 116
31 48
```

TSPA — worst route lists

Routes are listed using zero-based node indices; the first node is repeated at the end to show closure.

Random solution

100 selected nodes · 101 entries including closure

```
109 161 155 165 12 148 144 51 160 95 65 139 167 189 24 61
14 195 182 90 184 23 123 172 36 120 108 199 198 186 192
194 187 140 125 145 176 142 130 43 113 68 0 154 54 143 96
103 185 151 7 27 134 107 115 81 162 57 31 13 75 66 133 6
118 146 8 128 152 183 67 127 173 116 158 10 156 99 38 163
110 26 28 136 49 88 119 46 181 137 20 55 40 82 177 166 17
175 124 149 109
```

Nearest neighbor — end

100 selected nodes · 101 entries including closure

```
5 51 67 177 36 33 9 164 1 50 93 106 89 107 57 182 134 58
135 45 180 141 187 30 132 76 80 48 31 63 149 168 97 127
123 152 108 60 116 49 11 119 160 83 90 157 71 91 38 100 35
16 144 53 140 52 179 84 174 176 150 151 130 159 47 3 178
29 136 186 54 131 99 158 192 181 114 87 28 185 102 156 2
171 189 161 69 104 129 95 92 98 73 41 61 25 10 27 96 70 5
```

Nearest neighbor — flexible

100 selected nodes · 101 entries including closure

```
31 48 157 71 91 80 108 60 116 149 63 168 95 127 97 49 152
100 123 160 83 90 76 132 180 28 141 30 187 171 2 189 96 87
45 185 35 156 51 16 67 144 140 53 179 52 177 36 33 54 9
131 84 41 73 135 114 181 58 192 158 134 72 176 174 8 142
85 164 186 136 163 29 101 178 1 169 3 50 147 47 93 106 150
151 42 89 113 130 159 146 170 153 193 78 88 107 64 57 182
31
```

Greedy cycle

100 selected nodes · 101 entries including closure

```
18 16 52 33 9 54 174 8 142 164 186 136 163 29 101 178 1
169 3 50 147 47 93 106 42 89 113 130 159 146 170 153 193
78 88 107 64 57 182 151 150 176 72 131 36 177 179 53 140
41 84 134 158 192 58 181 114 135 73 45 144 67 35 156 185
102 28 87 141 189 2 171 187 30 180 90 83 160 123 100 152
49 97 127 168 63 149 116 60 108 31 48 80 91 157 71 4 132
76 51 18
```

TSPB — best route lists

Routes are listed using zero-based node indices; the first node is repeated at the end to show closure.

Random solution

100 selected nodes · 101 entries including closure

```
92 46 8 125 71 16 173 143 7 55 28 70 26 40 36 10 164 154
37 0 145 65 191 138 177 51 107 128 113 22 48 176 89 169 39
124 77 34 148 86 165 18 153 4 140 52 142 61 103 161 171 20
115 178 149 31 152 6 197 87 45 156 193 110 2 76 163 85 122
157 99 15 185 179 95 189 116 104 114 192 88 50 196 137 3
117 33 66 121 69 198 83 123 54 187 38 194 159 158 147 92
```

Nearest neighbor — end

100 selected nodes · 101 entries including closure

```
89 57 76 54 127 129 90 153 20 14 2 110 118 56 114 196 10
148 143 78 192 162 104 185 184 3 151 69 172 133 120 96 121
198 48 72 46 86 117 160 136 8 21 154 140 47 170 125 177 6
132 29 197 175 134 45 60 40 71 155 97 115 180 65 28 149
145 158 116 147 42 50 130 30 183 38 64 85 59 167 51 87 179
82 88 171 99 67 24 34 181 152 73 98 27 135 75 79 156 43 89
```

Nearest neighbor — flexible

100 selected nodes · 101 entries including closure

```
110 2 14 20 74 153 182 52 90 129 127 80 54 192 162 104 76
185 57 89 86 46 72 3 184 173 151 122 117 8 160 69 172 136
133 120 156 43 48 96 198 121 130 191 50 42 147 116 37 146
13 149 28 145 65 158 177 125 26 170 197 36 132 29 6 180
115 155 71 175 134 45 157 60 40 141 97 154 140 47 38 21
176 30 183 189 168 64 59 85 51 73 167 152 34 24 181 179 87
82 110
```

Greedy cycle

100 selected nodes · 101 entries including closure

```
174 36 197 6 180 115 170 97 140 47 155 71 175 134 45 157
60 141 40 33 64 21 189 168 69 160 172 136 133 120 8 117 72
46 86 184 3 173 122 151 98 111 27 135 76 185 162 192 80 52
182 20 74 2 93 110 118 14 153 90 129 127 54 104 57 89 156
43 48 96 198 121 63 130 191 183 30 176 38 154 26 125 177
138 158 145 149 147 42 116 37 146 13 28 65 178 103 29 132
1 174
```

TSPB — worst route lists

Routes are listed using zero-based node indices; the first node is repeated at the end to show closure.

Random solution

100 selected nodes · 101 entries including closure

```
26 13 173 63 41 121 183 162 45 144 72 29 73 169 44 64 131
94 112 125 12 123 9 174 39 117 37 116 167 161 11 158 34 22
130 4 32 103 77 25 30 133 59 157 129 180 99 195 138 146 79
78 75 154 18 1 38 2 128 83 52 60 100 28 68 85 109 33 43 71
58 110 82 24 168 108 166 54 141 14 124 185 188 95 49 47
111 199 164 113 23 55 139 53 17 176 67 74 86 10 26
```

Nearest neighbor — end

100 selected nodes · 101 entries including closure

```
14 20 2 110 90 129 127 54 192 162 76 57 89 96 121 198 120
69 172 133 136 160 184 3 151 59 85 167 51 87 179 82 88 171
99 67 24 34 181 152 73 40 60 71 170 154 140 47 38 21 30
183 130 191 116 149 28 145 65 158 125 177 6 132 29 197 175
134 45 157 155 97 115 180 36 1 174 178 103 19 26 64 33 141
142 62 91 113 5 102 53 55 138 146 37 13 147 42 50 195 14
```

Nearest neighbor — flexible

100 selected nodes · 101 entries including closure

```
110 68 118 93 9 2 188 14 20 74 17 171 39 99 163 31 67 181
126 88 179 87 82 167 51 101 85 107 59 73 25 152 34 24 11
66 119 84 150 153 182 52 90 129 127 80 106 192 54 159 162
104 144 76 135 75 185 79 57 83 89 32 86 46 72 109 137 3
184 173 27 22 111 98 151 122 95 117 8 160 69 21 30 183 189
168 172 136 133 120 92 166 156 43 164 48 96 198 121 58 110
```

Greedy cycle

100 selected nodes · 101 entries including closure

```
163 39 171 17 188 2 9 93 110 68 118 56 14 90 129 127 10
148 114 196 78 54 104 77 57 83 89 32 156 43 164 48 96 58
198 121 63 92 166 72 46 86 184 3 137 109 95 117 8 120 133
136 172 160 69 122 173 151 98 111 22 25 73 59 107 85 51
167 82 87 179 88 126 152 181 34 24 31 67 119 84 66 11 27
135 75 79 185 76 144 162 192 80 52 150 182 153 20 74 99
163
```

Conclusions and reproducibility

What this experiment shows

Greedy information matters

Random selection has a much wider and worse objective distribution. Using local objective changes to choose nodes produces substantially shorter and cheaper cycles.

The construction model matters

NN flexible and greedy cycle are related insertion heuristics, but they evaluate different partial solutions. The corrected implementation produces distinct solution spaces and results.

Practical takeaways

Strengths

- simple and interpretable
- valid solutions by construction
- objective-aware node selection
- reusable distance-matrix interface

Limitations

- greedy choices are irreversible
- insertion methods are expensive
- no local improvement phase
- random results vary by run

Reproducibility

The report is generated from the CSV statistics, route lists, and SVG visualisations stored in the evaluation directory. The same report generator can be reused for later laboratories by replacing the experiment data and method descriptions.

Source code: lab-01-greedy-heuristics/src/

Evaluation artifacts: lab-01-greedy-heuristics/evaluation/